

Matti Meikäläinen

Reliable Retry Handling in a Campus Event Service

An Illustrative ICT Master Thesis

Metropolia University of Applied Sciences

Master of Engineering

Information Technology

Master's Thesis

October 2026

PREFACE

This document was prepared as a teaching example for planning an ICT master thesis. Its purpose is to demonstrate the connection between a research question, a justified method, evidence and a limited conclusion.

Matti Meikäläinen is an example author, and Minna Hämäläinen is an example supervisor. No campus organization supplied data, approved a deployment or participated in the scenario.

Helsinki, October 2026
Matti Meikäläinen

Abstract

Author:	Matti Meikäläinen
Title:	Reliable Retry Handling in a Campus Event Service: An Illustrative ICT Master Thesis
Number of Pages:	23
Date:	October 2026
Degree:	Master of Engineering
Degree Programme:	Information Technology
Professional Major:	Software Engineering
Supervisors:	Minna Hämäläinen (Senior Lecturer)

Campus integrations were used as a fictional setting for studying event retries after lost responses. The example addressed the risk that a repeated request could create a second business record.

The illustrative objective was to compare a simple insertion service with a transactional deduplication design. The contribution was framed as a traceable comparison of correctness and latency under a bounded retry workload.

An engineering design study and a controlled comparison were described. Synthetic fixtures represented 1,000 unique events, 200 retries and five paired run-level latency summaries; no experiment or participant study was actually conducted.

In the synthetic example, duplicate records decreased from 200 to zero. Mean run-level p95 latency increased from 26 to 30 milliseconds, and payload conflicts were treated separately from valid retries.

The example illustrated a correctness–latency tradeoff rather than a general performance claim. Production effectiveness, external delivery guarantees and long-term behavior remained unestablished.

Keywords: idempotency, distributed systems, observability, validation

The originality of this thesis has been checked using Turnitin Originality Check service.

Tiivistelmä

Tekijä:	Matti Meikäläinen
Otsikko:	Luotettavat uudelleenyritykset kampuksen tapahtumapalvelussa
Sivumäärä:	23
Aika:	October 2026
Tutkinto:	Insinööri (ylempi AMK)
Koulutusohjelma:	Tieto- ja viestintätekniikka
Ammatillinen pääaine:	Ohjelmistotekniikka
Ohjaajat:	Minna Hämäläinen (Senior Lecturer)

Havainnollistavan työn tavoitteena oli parantaa uudelleenyritysten käsittelyä kuvitteellisessa kampuksen tapahtumapalvelussa. Tarkoituksena oli estää saman tapahtuman tallentuminen kahdesti tilanteessa, jossa lähettäjä ei saanut ensimmäisen pyynnön vastausta.

Esimerkissä kuvattiin tekninen suunnittelututkimus ja kontrolloitu vertailuasetelma, tietokantatapahtumaan perustuva ratkaisu ja toiminnallisten tarkistusten suunnitelma. Synteettinen aineisto sisälsi 1 000 yksilöllistä tapahtumaa, 200 uudelleenyritystä ja viisi parittaista viiveyhteenvetoa. Sovellusta tai vertailutestiä ei todellisuudessa suoritettu.

Havainnollistavassa aineistossa suojattu ratkaisu sisälsi 1 000 tapahtumariviä, kun vertailuratkaisu sisälsi 1 200 riviä. Ajokohtaisten p95-viiveiden keskiarvo kasvoi 26 millisekunnista 30 millisekuntiin. Ristiriitaiset hyötykuormat ja peruutustilanteet kuvattiin erillisinä tarkistuksina, ei onnistuneina testituloksina.

Esimerkki havainnollisti pysyvän tapahtumatunnisteen, rajatun tallennusvastuun ja jäljitettävien arviointikriteerien yhteyttä. Laskelmat eivät osoittaneet tuotantojärjestelmän suorituskykyä, rinnakkaisten pyyntöjen turvallisuutta tai ulkoisen toimituksen takeita. Nämä kysymykset jäivät todellisessa toteutuksessa arvioitaviksi.

Avainsanat:	uudelleenyritykset, hajautetut järjestelmät, havainnoitavuus, validointi

The originality of this thesis has been checked using Turnitin Originality Check service.

Contents

List of Abbreviations

1 Introduction 1
- 1.1 Context and Research Problem 1
- 1.2 Objective and Research Questions 1
- 1.3 Contribution, Scope and Delimitations 2
- 1.4 Method and Thesis Structure 2

2 Method and Material 3
- 2.1 Research Design 3
- 2.2 Reliability and Validity 3
- 2.3 Workload, Instrumentation and Analysis 4
- 2.4 Ethics and Reproduction Protocol 4

3 Current State Analysis and Project Specifications 6
- 3.1 Existing Workflow and Failure Modes 7
- 3.2 Derived Requirements and Acceptance Criteria 7
- 3.3 Constraints and Priorities 8

4 Theoretical Background 9
- 4.1 Retry Semantics and Application Identity 9
- 4.2 Durable Constraints and Transaction Boundaries 9
- 4.3 Observability and Diagnostic Evidence 10
- 4.4 Synthesis and Design Rationale 10

5 Results and Analysis 12
- 5.1 Architecture and Persistence Boundary 12
- 5.2 Implementation and Conflict Handling 14
- 5.3 Correctness Results and Failure Cases 15
- 5.4 Latency Analysis and Mathematical Model 16
- 5.5 Diagnostic Evidence and Requirement Traceability 17
- 5.6 Analysis of Tradeoffs and Unresolved Behavior 18

6 Discussions and Conclusions 19
- 6.1 Interpretation Against the Technical Framework 19
- 6.2 Explicit Answers to the Research Questions 19
- 6.3 Contribution and Practical Implications 20

6.4 Limitations, Reliability and Validity 20
6.5 Recommendations and Final Conclusions 20

7 Summary 22

References 24

A Appendix A. Synthetic Data and Test Specifications A-1
A.1 Run-Level Latency Fixture A-1
A.2 Extended Test Inventory A-1

B Appendix B. Technical Listings and Reproduction Material B-1
B.1 Payload, Client and Deployment Examples B-1
B.2 Protocol and Additional Language Boundaries B-2

C Appendix C. Authoring and Coverage Notes C-1
C.1 Supported Forms Demonstrated C-1

List of Abbreviations

The following abbreviations occur in the service scenario or its technical appendices. The first-use explanations in the chapters provide the context needed to understand each term.

API	Application programming interface
CI	Continuous integration
CSV	Comma-separated values
HTTP	Hypertext Transfer Protocol
ICT	Information and communications technology
ID	Identifier
JSON	JavaScript Object Notation
OTel	OpenTelemetry
RQ	Research question
SQL	Structured Query Language
SUT	System under test
UTC	Coordinated Universal Time

1 Introduction

The illustrative setting was a campus integration service receiving room-access and equipment-status events. A sender could lose the response after the server had already committed an event, making the outcome uncertain from the sender's perspective.

The engineering challenge was to distinguish a valid retransmission from a new event without silently accepting conflicting content. The thesis example investigated this challenge at the database transaction boundary rather than claiming to solve every failure mode in a distributed workflow.

1.1 Context and Research Problem

HTTP semantics distinguish idempotent methods from operations for which automatic repetition needs additional knowledge. This distinction motivated explicit application handling of repeated event submissions rather than an assumption that every POST could be repeated safely [1].

The fictional local gap was the absence of a documented link between retry identity, stored results and an observable audit trail. This was a scenario assumption, not a finding from interviews or a claim that the wider literature lacked retry-handling techniques. A real thesis would substantiate that gap with case evidence and a broader research review.

1.2 Objective and Research Questions

The example objective was to design and evaluate a transaction boundary that associated one accepted event with one stored outcome. The intended deliverable was a service design with a repeatable evaluation protocol and traceable acceptance criteria.

RQ1: Under the specified retry workload, how did transactional deduplication

affect duplicate event and outbox records compared with simple insertion? **RQ2:** What latency cost and diagnostic limitations accompanied that protection in the same controlled scenario?

1.3 Contribution, Scope and Delimitations

The proposed teaching contribution was a requirements-to-evidence framework for evaluating retry protection, including conflicting-payload cases. PostgreSQL and OpenTelemetry were existing tools; their inclusion alone was not presented as an original research contribution.

The scenario included one service, one database and tenant-scoped event identifiers. External message delivery, real personal data, multi-region replication and long-term operational effectiveness were excluded. These boundaries kept the comparison inspectable but prevented conclusions about an entire campus integration platform.

1.4 Method and Thesis Structure

The illustrative approach combined engineering design with a controlled comparison of two variants. Correctness criteria concerned duplicate persistence, conflict rejection and atomic event/outbox creation, while latency and trace completeness provided separate quality observations.

Chapter 2 defined the method and material, Chapter 3 derived specifications, and Chapter 4 compared the relevant technical approaches. Chapter 5 presented the illustrative artifact and synthetic evidence; Chapter 6 interpreted the questions and limitations, and Chapter 7 provided a factual summary. Appendices A–C retained example records, listings and authoring coverage.

2 Method and Material

The example adopted an engineering design study with a controlled, paired comparison of service variants. The design made the transaction policy the intended difference while keeping the event identities and retry schedule constant.

Synthetic data were selected to show how a method can be documented without exposing institutional records. This choice was suitable for a teaching document but supplied no evidence of how real users, devices or production systems behaved.

2.1 Research Design

RQ1 was addressed by comparing the number of persisted records with the number of unique accepted event identities. RQ2 was addressed by paired run-level p95 latency summaries and a review of which failure stages could be distinguished in the trace records.

A purely demonstrative walkthrough would have shown the interface but would not have established comparative behavior. A production case study could have added environmental realism, although uncontrolled workload and configuration differences would have required a different approach to attribution.

2.2 Reliability and Validity

For an actual execution, reliability would depend on versioned workloads, recorded configurations, stable timing boundaries and repeated runs. Reusing event IDs across variants would make paired comparisons possible, while randomized run order would reduce confounding from warm-up and resource contention.

Construct validity required counting business records rather than treating HTTP success as proof of correctness. External validity was restricted by the single-

node laboratory boundary. In this document, the stronger limitation was that all observations were invented fixtures, so no empirical validity or reliability was claimed.

2.3 Workload, Instrumentation and Analysis

The illustrative workload contained 1,000 unique valid events and 200 retransmissions of existing identities, yielding 1,200 requests. A separate conflict fixture reused an identity with a different payload; conflict cases were not mixed into the 1,200-request correctness denominator.

The latency material contained five paired run-level p95 values per variant at an illustrative arrival rate of 100 requests per second. Averaging these five summaries described run-to-run central tendency; it did not recover the pooled request-level p95. The retained CSV in Appendix A made this distinction explicit.

2.4 Ethics and Reproduction Protocol

The conceptual diagram separates specification coverage from observed evidence. Their overlap represents checks that have both a defined criterion and an auditable outcome; it does not quantify coverage in this synthetic example.

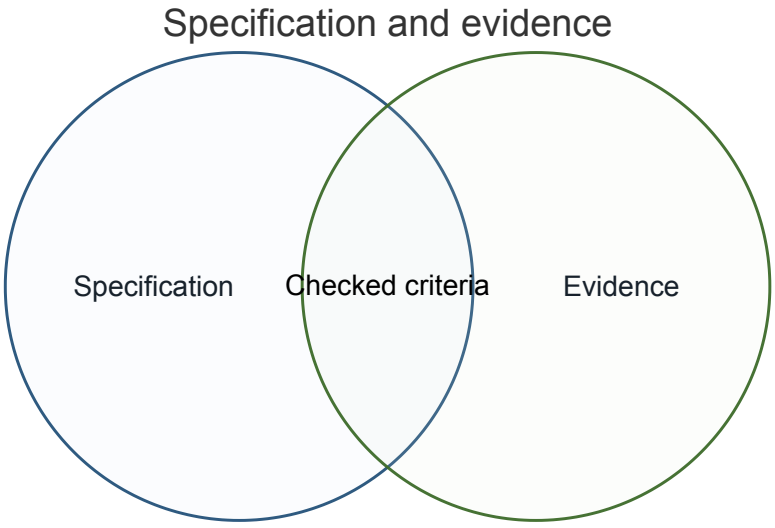


Figure 1. Conceptual overlap between specifications and evidence.

An item in either circle alone would require further work before it could support a validated claim. This diagram is a conceptual illustration rather than a measurement.

No participants, institutional records or production endpoints were used. Tenant IDs and event IDs were fictional, and examples contained no authentication secrets or personal attributes. No ethics approval, consent process or organizational endorsement was asserted.

A real run would record the service revision, database version, schema migration, container digest and workload seed before starting. The protocol would warm each variant, apply the identical workload, export raw measurements, and compare database invariants with the expected identity set. The commands in Appendix B illustrated that protocol and were not executed to produce the synthetic results.

3 Current State Analysis and Project Specifications

The following sequence illustrates the intended handling of a repeated event. It represents the protected design's expected interaction, not a captured execution trace.

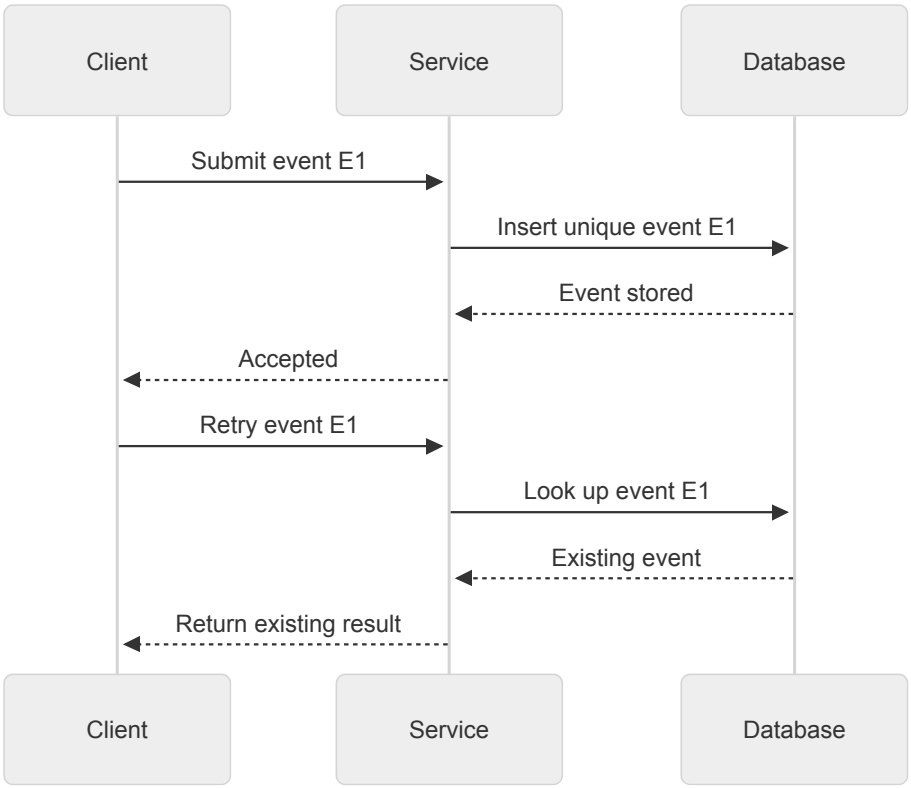


Figure 2. Illustrative handling of an initial event and a retry.

The second submission should reuse the existing identity. Transaction and concurrency guarantees still require the checks described later in the thesis.

The scenario assumed that a campus sender retried an event when no response arrived within its timeout. The baseline stored every accepted submission with a new internal row identifier, even if the sender supplied an identity already encountered.

The analysis separated uncertainty about transport from uncertainty about per-

sistence. A timeout established that a response was not observed; it did not establish that the original transaction had failed. This distinction determined which observations were needed during validation.

3.1 Existing Workflow and Failure Modes

In the invented workflow, a room device submitted an event through an application programming interface (API). A downstream process later consumed an outbox record. Staff needed a consistent event history and enough diagnostic information to explain repeated submissions.

Three scenario failure points were considered: rejection before persistence, a database rollback, and loss of the response after commit. The third case exposed the baseline weakness because a repeated request could be processed as a second business event. These were constructed cases, not incidents observed at a university.

3.2 Derived Requirements and Acceptance Criteria

The requirements prioritized durable identity over a memory-only cache because the example included service restarts. Identity was scoped by tenant so that unrelated organizations could use the same event label without unintentionally sharing a deduplication boundary.

Table 1 connected each scenario need to an explicit check. The acceptance criteria were design choices for this example, not universal service-level targets.

Table 1. Scenario requirements and acceptance criteria.

ID	Requirement	Acceptance criterion
R1	Suppress repeated valid identity	One event and one outbox row per tenant/event pair
R2	Detect conflicting payload	Reject changed content for an existing identity

ID	Requirement	Acceptance criterion
R3	Preserve atomic persistence	Event and outbox both commit or both roll back
R4	Expose outcome stage	Record accepted, replayed, conflict or failed status
R5	Quantify latency tradeoff	Compare five paired run-level p95 summaries

The matrix kept diagnostic completeness separate from correctness. A useful trace could explain a failure without proving that duplicate business effects were impossible.

3.3 Constraints and Priorities

The example allowed one PostgreSQL transaction for the event and outbox tables and required a durable key after restart. It did not assume that a remote consumer participated in that transaction, so end-to-end delivery semantics remained outside the established boundary.

Correctness was prioritized over a small increase in latency in the synthetic decision framework. Chapter 4 therefore examined identity semantics, database enforcement and observability separately, rather than treating a faster response or a richer dashboard as a substitute for persistence guarantees.

4 Theoretical Background

The conceptual framework separated request semantics, persistence constraints and diagnostic visibility. These categories matched the HTTP, database and telemetry documentation used as technical anchors [1, 2, 3].

The comparison asked which mechanism addressed which requirement and where each mechanism stopped. This was the example author's analytical organization, not a claim that three documentation pages constituted a complete state-of-the-art review.

4.1 Retry Semantics and Application Identity

RFC 9110 defined idempotency by the intended effect of repeated identical requests. It also distinguished repeatable methods from operations whose safe repetition required additional knowledge, which justified an explicit retry policy for the example POST interface [1].

The application policy used a tenant/event pair and a payload digest, rather than transport connection identity. This was a design inference for the scenario: the same event could arrive through a new connection, while a reused ID with changed content represented a conflict rather than a valid retransmission.

4.2 Durable Constraints and Transaction Boundaries

PostgreSQL unique constraints restricted repeated values in a column or group of columns. Its documentation described automatic unique-index creation and the significance of null handling, motivating non-null identity fields in the illustrative schema [2].

A memory-only cache offered a different boundary: it could avoid a lookup during one process lifetime but did not itself establish restart-safe uniqueness. The example therefore compared the cache as an alternative rather than presenting

it as equivalent to a durable database constraint.

4.3 Observability and Diagnostic Evidence

OpenTelemetry described traces, metrics and logs as distinct signals for observing system activity. These categories supported the example's separation of request path, latency summaries and event-status records [3].

The design interpretation was that correlation could help identify whether a retry reached persistence, while table invariants remained the correctness evidence. A complete trace was not proof that a remote downstream effect occurred only once, and a missing trace was not proof that persistence failed.

4.4 Synthesis and Design Rationale

Table 2 compared three options against the scenario requirements. The comparison was qualitative and concerned the specified failure model; it did not rank database products or benchmark alternative architectures.

A durable identity constraint plus atomic event/outbox persistence was selected for the example because it directly exposed the R1–R3 invariants. Telemetry remained complementary evidence for R4, while R5 required an explicit latency comparison.

Table 2. Analytical comparison within the fictional scenario.

Criterion	Simple insertion	Process-local cache	Durable key + transaction
Repeated identity	New row	May replay while cached	Database-enforced boundary
Process restart	No retained protection	Cache state lost	Key retained
Payload conflict	Not distinguished	Additional logic needed	Explicit digest comparison

Criterion	Simple insertion	Process-local cache	Durable key + transaction
Remote delivery	Not established	Not established	Not established

None of these options alone established the behavior of an external consumer. The selected design was therefore evaluated for local persistence, not described as a universal exactly-once delivery solution.

5 Results and Analysis

The example artifact placed identity checking and outbox insertion within one persistence boundary. The contribution was illustrated through an evidence matrix that distinguished duplicate suppression, conflict handling, atomicity and diagnostic visibility.

The chapter followed design, implementation and evaluation in that order. Figures represented the proposed system, code illustrated selected boundaries, and tables reported invented observations whose arithmetic could be inspected independently.

5.1 Architecture and Persistence Boundary

Figure 1 showed the client, API service, database and telemetry collector. The collector was drawn outside the transaction boundary because diagnostic export was not part of the event/outbox commit.

The service associated a stored result with each accepted tenant/event pair. A successful replay returned that stored outcome, while an attempt to reuse the identity for different content entered the conflict path.

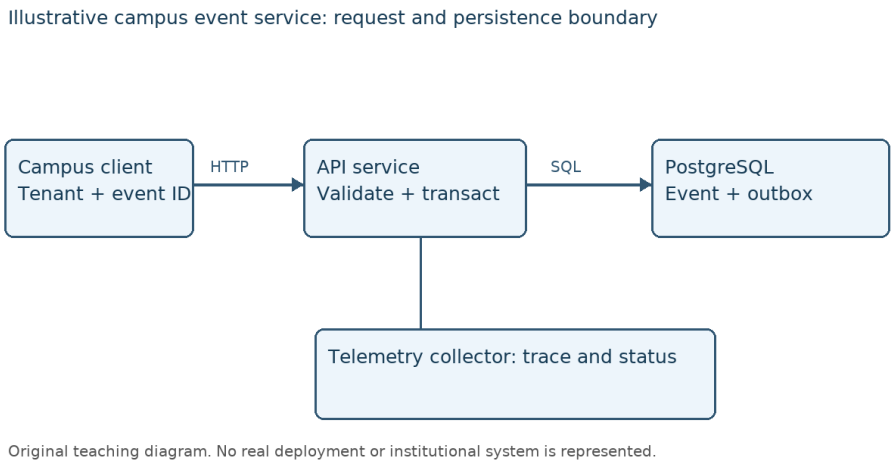


Figure 3. Proposed service and local persistence boundary; original teaching diagram.

The diagram made the database the decision point for durable identity. It deliberately omitted remote delivery acknowledgment, whose guarantees were not evaluated in this example.

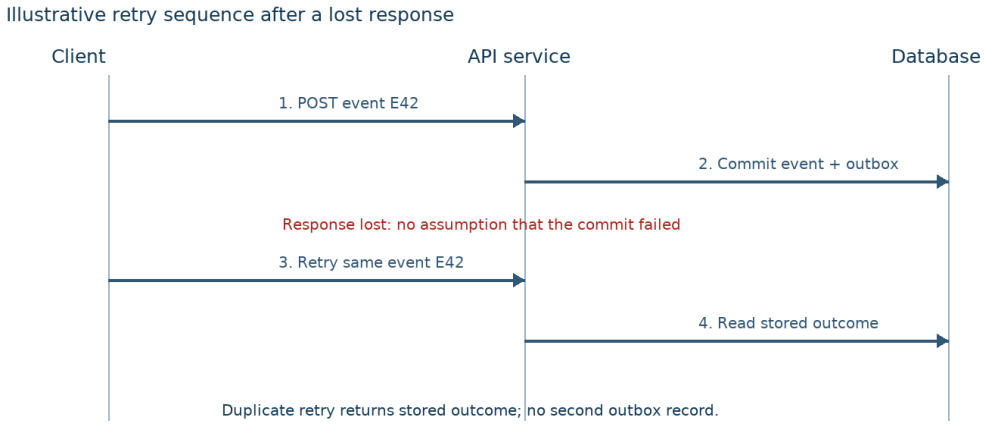


Figure 4. Lost-response retry sequence; original teaching diagram.

Figure 2 followed a commit whose response was lost. The second request was a replay of the same logical event, so an absent first response did not justify a second outbox record.

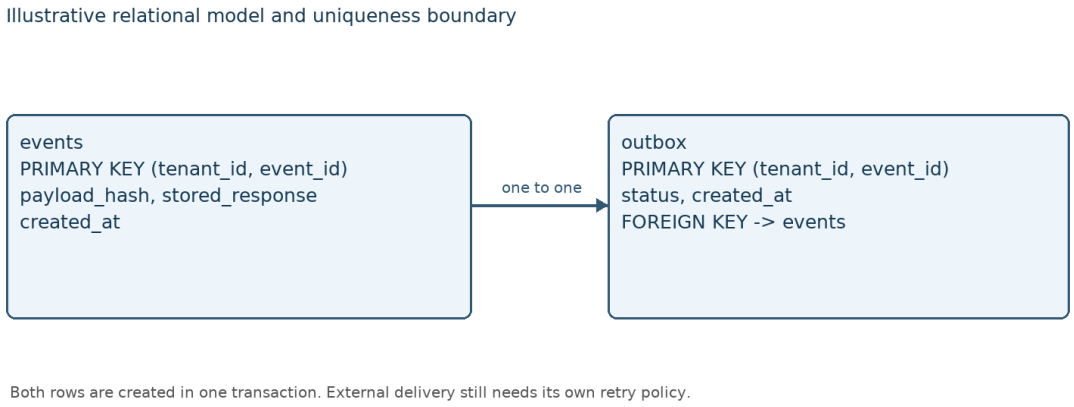


Figure 5. Event and outbox identity relationship; original teaching diagram.

Figure 3 expressed a one-to-one event/outbox relationship inside the selected transaction. It did not imply that any external consumer shared this transaction.

5.2 Implementation and Conflict Handling

Listing 1 expressed the control flow using an abstract transaction adapter. The adapter was assumed to enforce uniqueness and atomic event/outbox creation; the listing was pseudocode and was not an executable implementation of database concurrency.

A digest compared the canonical payload with the stored content identity. A production implementation would specify canonicalization, bounded key retention, transaction isolation and handling of concurrent insertion conflicts; those details could not be inferred from the short extract alone.

Listing 1. Transaction-adapter pseudocode for event acceptance.

```
def accept_event(tx, tenant_id, event_id, digest, payload):
    # Pseudocode: the adapter must handle concurrent unique conflicts.
    with tx.atomic():
        previous = tx.find_event(tenant_id, event_id)
        if previous is not None:
            if previous.digest != digest:
                return {"status": 409, "outcome": "conflict"}
            return previous.response
        response = {"status": 201, "event_id": event_id}
        tx.insert_event(tenant_id, event_id, digest, response)
        tx.insert_outbox(tenant_id, event_id, payload)
        return response
```

The extract separated replay from payload conflict. Its correctness depended on the adapter contract and would require concurrent and rollback testing in a real implementation.

The SQL extract below showed the identity constraint for the event table. Appendix B supplied related payload and configuration examples.

Listing 2. Illustrative non-null event identity constraint.

```
CREATE TABLE events (
    tenant_id text NOT NULL,
    event_id text NOT NULL,
    payload_hash text NOT NULL,
    stored_response jsonb NOT NULL,
    PRIMARY KEY (tenant_id, event_id)
);
```

The compound key made identity tenant-specific. The outbox foreign key and its unique event association were separate schema requirements described by Figure 3, not silently supplied by this short table definition.

5.3 Correctness Results and Failure Cases

The correctness fixture assigned 1,000 unique event identities and 200 repeated valid requests to each variant. Simple insertion was assigned 1,200 event rows and 1,200 outbox rows, while the protected variant was assigned 1,000 of each.

The 200 excess baseline rows represented duplicate persistence within the constructed fixture. Table 3 showed the arithmetic and did not establish that either variant had been run. Separate conflict, rollback and concurrent-retry cases were retained as example test specifications in Appendix A.

Table 3. Synthetic correctness fixture; no tests were executed.

Observation	Baseline	Protected
Unique valid identities	1000	1000
Valid requests including retries	1200	1200
Event rows	1200	1000
Outbox rows	1200	1000
Excess rows per table	200	0

The fixture supported a teaching comparison of the intended invariants. It did not show resilience under crashes, isolation anomalies or remote-consumer retries, which remained specification cases rather than observed successes.

5.4 Latency Analysis and Mathematical Model

Five synthetic paired summaries represented run-level p95 latency at the same illustrative arrival rate. Baseline values were 22, 24, 25, 27 and 32 milliseconds; protected values were 25, 27, 29, 31 and 38 milliseconds, as retained in the CSV attachment.

For variant v, $L_{v,i}$ denoted the p95 latency in run i, measured in milliseconds, and $n = 5$ denoted the number of runs. Equation 1 defined the mean of those run-level summaries; it was not the percentile of all requests pooled together.

$$\bar{L}_v = \frac{1}{n} \sum_{i=1}^{n} L_{v,i} \tag{1}$$

The mean was 26 milliseconds for the baseline and 30 milliseconds for the protected fixture. Equation 2 expressed their relative difference, where Δ_L was dimensionless before multiplication by 100.

$$\Delta_L = \frac{\bar{L}_{\text{protected}} - \bar{L}_{\text{baseline}}}{\bar{L}_{\text{baseline}}} \times 100\% = \frac{30 - 26}{26} \times 100\% \approx 15.4\% \tag{2}$$

Figure 4 displayed the same five pairs rather than a different dataset. The visibly larger protected values illustrated a possible cost of additional persistence work, without establishing a causal effect in real systems.

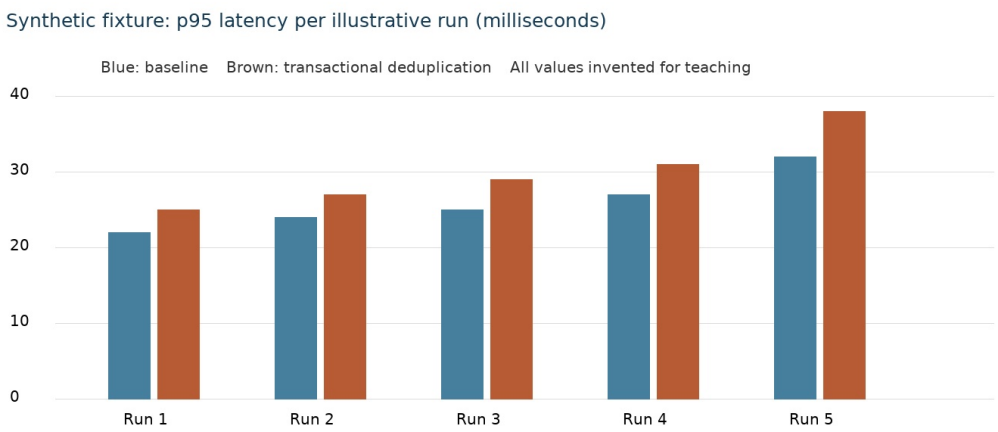


Figure 6. Invented run-level p95 latency values; not benchmark measurements.

The mean paired difference was 4 milliseconds. With no measured request distribution, the example did not report confidence intervals, significance tests or a claim about a production latency objective.

5.5 Diagnostic Evidence and Requirement Traceability

The illustrative trace used one event identity across an initial acceptance and a replay. The status fields described the service's decision, while the durable record count remained the required persistence check.

The trace below was hand-authored teaching material. It demonstrated a readable log listing, not proof that instrumentation had observed the example transaction.

Listing 3. Synthetic status trace for a lost-response replay.

```
2026-10-27T10:00:00Z tenant=t1 event=E42 outcome=accepted
2026-10-27T10:00:01Z tenant=t1 event=E42 outcome=replayed
2026-10-27T10:00:02Z tenant=t1 event=E42 outcome=conflict
```

Table 4. Evidence matrix and explicit claim boundary.

Criterion	Evidence type	Fixture outcome	Established empirically
R1 / RQ1	Table 3 arithmetic	Zero protected excess rows	No; synthetic
R2	Conflict test specific ation	Reject changed digest	No; not executed
R3	Transaction design	Atomic event/outbo x intent	No; not executed
R4 / RQ2	Hand-authored trace	Status categories visible	No; synthetic
R5 / RQ2	Five CSV pairs	Mean difference 4 ms	No; synthetic

The matrix prevented specification coverage from being mistaken for successful execution. A real study would replace the final column with auditable outcomes and links to raw evidence.

5.6 Analysis of Tradeoffs and Unresolved Behavior

The constructed comparison illustrated why lower duplicate persistence could be evaluated alongside higher latency rather than collapsed into a single success label. It also showed why a replay response and an atomic database outcome were different observations.

Concurrent requests, retention expiry and downstream redelivery remained unresolved in the evidence. The pseudocode’s lookup-then-insert shape particularly required a real adapter to resolve unique-key races; no conclusion of concurrency safety followed from the diagram or the synthetic table.

6 Discussions and Conclusions

The example addressed ambiguity after a lost response by making accepted identity durable and observable. The central lesson was the need to establish the persistence boundary explicitly before drawing conclusions about retries.

The results chapter intentionally distinguished a design claim, a calculated fixture and an executed observation. That distinction limited the conclusion more strongly than the attractive zero-duplicate value in the example table.

6.1 Interpretation Against the Technical Framework

The proposed boundary was consistent with the earlier separation of HTTP retry semantics, database identity enforcement and telemetry. In the example analysis, these mechanisms answered different questions and none replaced the other two.

The protected fixture illustrated the anticipated correctness–latency tradeoff, but there was no empirical comparison with published performance results. The short technical review could motivate an experiment; it could not establish that this design advanced the state of the art.

6.2 Explicit Answers to the Research Questions

RQ1: Within the invented fixture, transactional deduplication was represented by 1,000 event and outbox rows rather than 1,200, eliminating the 200 excess rows assigned to simple insertion. This answered the scenario arithmetic but did not establish observed duplicate suppression or concurrency safety.

RQ2: The five synthetic run-level summaries yielded a 4-millisecond mean increase, approximately 15.4 percent relative to the baseline mean. The hand-authored trace illustrated accepted, replayed and conflict decisions, while actual instrumentation completeness and production latency remained unmeasured.

6.3 Contribution and Practical Implications

The illustrative contribution was the explicit pairing of identity invariants with an evidence matrix and a bounded latency comparison. Its value as teaching material was that a reader could see which statements required database counts, trace records or additional experiments.

No validated contribution to distributed-systems research was claimed. For a real campus project, the framework could organize acceptance criteria and help reviewers request missing evidence, but its operational effectiveness would have to be established by actual execution and external review.

6.4 Limitations, Reliability and Validity

The most important constraint was the absence of executed measurements: every correctness and latency value was invented. Consequently, the example could establish arithmetic consistency and document structure, but not service performance, reliability or a causal effect of the design.

The single-service boundary omitted external delivery and long-term identifier retention. This prevented an end-to-end delivery claim and left the effect of retention expiry unresolved. A real study would add crash injection, concurrent clients, measured request distributions and a defined retention policy before making stronger claims.

6.5 Recommendations and Final Conclusions

The first recommendation for an actual study was to implement and test the transaction adapter before optimizing latency. A second was to retain workload and schema versions with database-invariant checks, and to investigate concurrent insertion and rollback behavior independently of the happy path.

The final conclusion of the illustrative document was methodological: retry

correctness required an explicit identity boundary and evidence matched to the claimed guarantee. The document demonstrated how to present that argument, while leaving all real engineering and empirical claims open for future investigation.

7 Summary

The illustrative thesis considered reliable retry handling in a fictional campus event service. The starting problem was an uncertain request outcome after loss of a response: an event might already have committed even though the sender did not receive confirmation. The objective was to frame a comparison between simple insertion and a durable, tenant-scoped deduplication policy.

Two research questions organized the example. RQ1 concerned duplicate event and outbox persistence under a defined retry workload, while RQ2 concerned latency cost and diagnostic limitations. The intended contribution was a traceable framework linking identity requirements to an explicit persistence boundary and an evidence matrix, rather than the invention of a database or telemetry system.

The described method combined engineering design with a controlled paired comparison. The workload fixture represented 1,000 unique valid events and 200 retransmissions, with conflict cases specified separately. Five paired run-level p95 latency summaries and a hand-authored status trace demonstrated how material and analysis could be documented. No implementation benchmark, participant study or production deployment was conducted.

The proposed artifact associated one stored outcome with a tenant/event identity and paired the event with an outbox record inside one transaction. A matching replay returned the stored outcome, while changed content for an existing identity entered a conflict path. The short Python listing expressed an adapter contract and did not constitute an executed, concurrency-safe database implementation.

In the synthetic correctness table, the baseline contained 1,200 event rows and the protected variant contained 1,000, corresponding to 200 and zero excess rows respectively. The invented latency summaries averaged 26 milliseconds for the baseline and 30 milliseconds for the protected variant. Their mean paired difference was 4 milliseconds, about 15.4 percent of the baseline mean; this calculation was not a pooled request-level percentile.

The example trace illustrated accepted, replayed and conflicting outcomes. Its categories supplied a model for diagnostic presentation, while durable counts remained the relevant correctness observation. The evidence matrix separated these fixtures from unexecuted conflict, rollback and concurrency cases so that coverage could not be mistaken for passed tests.

The discussion answered both questions only within the constructed example. No empirical performance, production reliability, end-to-end delivery or scientific novelty was established. The conclusions emphasized matching evidence to a bounded guarantee and stated that concurrency, retention expiry and external consumption remained unresolved. The document therefore served as an annotated model of research presentation rather than a completed master research study.

References

[1] Fielding R, Nottingham M, Reschke J. HTTP Semantics. RFC 9110. Internet Engineering Task Force; 2022. Available from: https://www.rfc-editor.org/rfc/rfc9110.html

[2] PostgreSQL Global Development Group. PostgreSQL 18 Documentation: Constraints, Section 5.5. Available from: https://www.postgresql.org/docs/18/ddl-constraints.html [accessed 27 October 2026].

[3] OpenTelemetry Authors. Signals. OpenTelemetry documentation. Available from: https://opentelemetry.io/docs/concepts/signals/ [accessed 27 October 2026].

A Appendix A. Synthetic Data and Test Specifications

Chapter 5 referred to this appendix for the run-level latency values and detailed test cases. The CSV values could reproduce the example arithmetic, while the test inventory specified what a real implementation would need to check.

A.1 Run-Level Latency Fixture

The retained file contained one row per illustrative run, with paired p95 latency values in milliseconds. Its five pairs were identical to the values in Chapter 5 and Figure 4.

The mean of the baseline column was 26 and the mean of the protected column was 30. These column means summarized five run-level percentiles; request-level distributions were not retained because no requests were measured.

Listing 4. Synthetic paired summaries retained as CSV.

```
run,baseline_p95_ms,protected_p95_ms
1,22,25
2,24,27
3,25,29
4,27,31
5,32,38
```

A.2 Extended Test Inventory

The inventory varied failure condition and tenant fixture across twelve groups. Six scenarios per group created 72 specification rows, deliberately exercising multi-page table rendering.

Every row remained not executed. Repetition across groups illustrated how a test inventory could preserve explicit identifiers without manufacturing passing outcomes.

Table 5. Detailed specification inventory; every case is unexecuted.

Case	Need	Scenario	Status
T01-1	R1	Tenant fixture 1: Same identity, same digest	Not executed
T01-2	R2	Tenant fixture 1: Same identity, changed digest	Not executed
T01-3	R1	Tenant fixture 1: Same event ID, different tenant	Not executed
T01-4	R3	Tenant fixture 1: Rollback before outbox insert	Not executed
T01-5	R3	Tenant fixture 1: Concurrent identical insertions	Not executed
T01-6	R4	Tenant fixture 1: Response lost after commit	Not executed
T02-1	R1	Tenant fixture 2: Same identity, same digest	Not executed
T02-2	R2	Tenant fixture 2: Same identity, changed digest	Not executed
T02-3	R1	Tenant fixture 2: Same event ID, different tenant	Not executed
T02-4	R3	Tenant fixture 2: Rollback before outbox insert	Not executed
T02-5	R3	Tenant fixture 2: Concurrent identical insertions	Not executed
T02-6	R4	Tenant fixture 2: Response lost after commit	Not executed
T03-1	R1	Tenant fixture 3: Same identity, same digest	Not executed
T03-2	R2	Tenant fixture 3: Same identity, changed digest	Not executed
T03-3	R1	Tenant fixture 3: Same event ID, different tenant	Not executed

Case	Need	Scenario	Status
T03-4	R3	Tenant fixture 3: Rollback before outbox insert	Not executed
T03-5	R3	Tenant fixture 3: Concurrent identical insertions	Not executed
T03-6	R4	Tenant fixture 3: Response lost after commit	Not executed
T04-1	R1	Tenant fixture 4: Same identity, same digest	Not executed
T04-2	R2	Tenant fixture 4: Same identity, changed digest	Not executed
T04-3	R1	Tenant fixture 4: Same event ID, different tenant	Not executed
T04-4	R3	Tenant fixture 4: Rollback before outbox insert	Not executed
T04-5	R3	Tenant fixture 4: Concurrent identical insertions	Not executed
T04-6	R4	Tenant fixture 4: Response lost after commit	Not executed
T05-1	R1	Tenant fixture 5: Same identity, same digest	Not executed
T05-2	R2	Tenant fixture 5: Same identity, changed digest	Not executed
T05-3	R1	Tenant fixture 5: Same event ID, different tenant	Not executed
T05-4	R3	Tenant fixture 5: Rollback before outbox insert	Not executed
T05-5	R3	Tenant fixture 5: Concurrent identical insertions	Not executed
T05-6	R4	Tenant fixture 5: Response lost after commit	Not executed
T06-1	R1	Tenant fixture 6: Same identity, same digest	Not executed

Case	Need	Scenario	Status
T06-2	R2	Tenant fixture 6: Same identity, changed digest	Not executed
T06-3	R1	Tenant fixture 6: Same event ID, different tenant	Not executed
T06-4	R3	Tenant fixture 6: Rollback before outbox insert	Not executed
T06-5	R3	Tenant fixture 6: Concurrent identical insertions	Not executed
T06-6	R4	Tenant fixture 6: Response lost after commit	Not executed
T07-1	R1	Tenant fixture 7: Same identity, same digest	Not executed
T07-2	R2	Tenant fixture 7: Same identity, changed digest	Not executed
T07-3	R1	Tenant fixture 7: Same event ID, different tenant	Not executed
T07-4	R3	Tenant fixture 7: Rollback before outbox insert	Not executed
T07-5	R3	Tenant fixture 7: Concurrent identical insertions	Not executed
T07-6	R4	Tenant fixture 7: Response lost after commit	Not executed
T08-1	R1	Tenant fixture 8: Same identity, same digest	Not executed
T08-2	R2	Tenant fixture 8: Same identity, changed digest	Not executed
T08-3	R1	Tenant fixture 8: Same event ID, different tenant	Not executed
T08-4	R3	Tenant fixture 8: Rollback before outbox insert	Not executed
T08-5	R3	Tenant fixture 8: Concurrent identical insertions	Not executed

Case	Need	Scenario	Status
T08-6	R4	Tenant fixture 8: Response lost after commit	Not executed
T09-1	R1	Tenant fixture 9: Same identity, same digest	Not executed
T09-2	R2	Tenant fixture 9: Same identity, changed digest	Not executed
T09-3	R1	Tenant fixture 9: Same event ID, different tenant	Not executed
T09-4	R3	Tenant fixture 9: Rollback before outbox insert	Not executed
T09-5	R3	Tenant fixture 9: Concurrent identical insertions	Not executed
T09-6	R4	Tenant fixture 9: Response lost after commit	Not executed
T10-1	R1	Tenant fixture 10: Same identity, same digest	Not executed
T10-2	R2	Tenant fixture 10: Same identity, changed digest	Not executed
T10-3	R1	Tenant fixture 10: Same event ID, different tenant	Not executed
T10-4	R3	Tenant fixture 10: Rollback before outbox insert	Not executed
T10-5	R3	Tenant fixture 10: Concurrent identical insertions	Not executed
T10-6	R4	Tenant fixture 10: Response lost after commit	Not executed
T11-1	R1	Tenant fixture 11: Same identity, same digest	Not executed
T11-2	R2	Tenant fixture 11: Same identity, changed digest	Not executed
T11-3	R1	Tenant fixture 11: Same event ID, different tenant	Not executed

Case	Need	Scenario	Status
T11-4	R3	Tenant fixture 11: Rollback before outbox insert	Not executed
T11-5	R3	Tenant fixture 11: Concurrent identical insertions	Not executed
T11-6	R4	Tenant fixture 11: Response lost after commit	Not executed
T12-1	R1	Tenant fixture 12: Same identity, same digest	Not executed
T12-2	R2	Tenant fixture 12: Same identity, changed digest	Not executed
T12-3	R1	Tenant fixture 12: Same event ID, different tenant	Not executed
T12-4	R3	Tenant fixture 12: Rollback before outbox insert	Not executed
T12-5	R3	Tenant fixture 12: Concurrent identical insertions	Not executed
T12-6	R4	Tenant fixture 12: Response lost after commit	Not executed

The table was a specification artifact. Its length and number of cases did not establish coverage of all failure modes or successful system behavior.

B Appendix B. Technical Listings and Reproduction Material

The following extracts supplemented the transaction-boundary discussion in Chapter 5. They demonstrated several ICT document forms without claiming to compose a runnable service.

All addresses, image names and identifiers were examples. The thesis pipeline displayed the listings but did not execute the deployment or load commands.

B.1 Payload, Client and Deployment Examples

The payload carried a tenant-scoped logical identity and simple event content. The JavaScript client retained the same event ID for a replay; generating a new ID for each attempt would evade the intended deduplication boundary.

The configuration illustrated where telemetry and database endpoints would be supplied. It omitted credentials, health checks and production hardening and therefore was not a deployable manifest.

Listing 5. Illustrative event payload.

```
{
  "tenant_id": "campus-demo",
  "event_id": "E42",
  "event_type": "equipment.ready",
  "payload": {"device_id": "demo-7"}
}
```

Listing 6. Client extract preserving logical event identity.

```javascript
async function submitEvent(event) {
  const response = await fetch("/events", {
    method: "POST",
    headers: {"Content-Type": "application/json"},
    body: JSON.stringify(event)
  });
  return {status: response.status, body: await response.json()};
}
```

Listing 7. Illustrative service configuration; example image only.

```yaml
services:
  api:
    image: example.invalid/campus-events:demo
    environment:
      DB_HOST: database
      OTEL_SERVICE_NAME: campus-events-demo
  database:
    image: postgres:18
```

The client extract omitted retry timing and authentication. The deployment fragment only illustrated configuration syntax and did not contain the service implementation or database initialization.

B.2 Protocol and Additional Language Boundaries

A real reproduction protocol would pin all revisions, validate the environment and archive raw output. The Bash extract showed command structure only; the referenced benchmark program was hypothetical and was not included or executed.

The Java and C++ extracts illustrated a shared boundary rule for nonempty event identifiers. They were optional interoperability examples rather than independently evaluated components of the artifact.

Listing 8. Unexecuted reproduction command structure.

```
# Illustrative only: benchmark program is not supplied.
docker compose up -d
curl --fail http://localhost:8080/health
python benchmark.py --seed 42 --events 1000 --retries 200
# Archive raw results and check database invariants separately.
```

Listing 9. Optional Java identifier boundary example.

```
public boolean validEventId(String eventId) {
    return eventId != null && !eventId.trim().isEmpty();
}
```

Listing 10. Optional edge-client identifier boundary example.

```
#include <string>
bool valid_event_id(const std::string& id) {
    return id.find_first_not_of(" ") != std::string::npos;
}
```

The two identifier checks were intentionally minimal and did not establish equivalent Unicode whitespace handling or a full validation policy. A real interoperability claim would require a shared specification and boundary tests.

C Appendix C. Authoring and Coverage Notes

This fictional example demonstrates supported document forms. Its topic, narrative and numerical results are synthetic; they are not completed research.

The cover, contents and numbering were generated by the existing pipeline. The examples exercised supported forms, not every possible discipline-specific artifact or every university formatting rule.

C.1 Supported Forms Demonstrated

The main text demonstrated captions, citations, bold and italic text, inline code, paired quantitative comparison, inline mathematics and numbered display equations. Appendices demonstrated attached source files, configuration languages and a repeated-header table.

The blockquote below was an authored synthesis rather than a quotation from an external source. An actual quotation would need precise source attribution and a locator under the applicable rules.

A timeout described uncertainty at the caller; it did not establish that a transaction had rolled back.

The synthesis illustrated the italic blockquote style. It introduced no additional empirical evidence.

Table 6. Authoring coverage and location.

Form	Location
Abstract translations	English and Finnish
Black worked paragraphs	All seven chapters
PNG diagrams and JPEG chart	Chapter 5
Short tables and weighted columns	Chapters 3–5

Form	Location
Multi-page table and repeated header	Appendix A
Python, SQL, JSON and JavaScript	Chapter 5 and Appendix B
YAML, Bash, Java, C++ and text	Appendices A–B
Inline math and numbered equations	Section 5.4
Source attachments	Python and CSV listings
Citation and real reference entries	Chapters 1 and 4; References
Bold, italic, inline code and quote	Throughout; this appendix
Ordered and unordered lists	Following examples

The following ordered list illustrated a compact review sequence. Its steps were teaching recommendations, not records of completed research.

1. Identify the guarantee being claimed.
2. Match each guarantee to an observable invariant.
3. Check the evidence and state remaining uncertainty.

The unordered list separated excluded document forms from supported examples. These boundaries should inform authoring choices.

- Merged table cells and automatic landscape pages were not demonstrated.
- Nested lists and full citations inside tables were not relied upon.
- The Finnish abstract is enabled for this example; the Swedish abstract is disabled.